

Stored Procedures and Triggers in MySQL

— Beginner to Advanced

Think of it this way:

Feature	Stored Procedure	Trigger
What is it?	A reusable SQL program that runs when called	A SQL program that runs automatically when an event occurs
Executed by	User/Application	Database automatically
Called using	<code>CALL procedure_name()</code>	No explicit call
Used for	Business logic, reports, ETL, validations	Auditing, logging, automatic updates
Parameters	Yes	No
Return values	Output parameters	No

PANEL 1 — Sample Tables

We'll use these tables throughout all examples.

```
CREATE TABLE employees
(
  emp_id INT PRIMARY KEY AUTO_INCREMENT,
  emp_name VARCHAR(100),
  department VARCHAR(50),
  salary DECIMAL(10,2),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
INSERT INTO employees(emp_name,department,salary)
```

```
VALUES  
( 'John','IT',50000),  
( 'Mary','HR',60000),  
( 'David','IT',70000),  
( 'Lisa','Finance',80000);
```

Check data:

```
SELECT * FROM employees;
```

PART 1 — STORED PROCEDURES

PANEL 2 — Beginner Level

What is a Stored Procedure?

A stored procedure is a group of SQL statements saved inside the database.

Instead of writing:

```
SELECT * FROM employees;
```

every time,

you create it once and call it whenever needed.

Example 1: Procedure Without Parameters

Create Procedure

```
DELIMITER $$
```

```
CREATE PROCEDURE sp_get_employees()  
BEGIN  
    SELECT *  
    FROM employees;  
END $$
```

DELIMITER ;

Execute Procedure

```
CALL sp_get_employees();
```

Output

```
1 John IT 50000
2 Mary HR 60000
3 David IT 70000
4 Lisa Finance 80000
```

Explanation

```
CREATE PROCEDURE sp_get_employees()
```

Creates a procedure named:

```
sp_get_employees
BEGIN
...
END
```

Procedure body.

```
CALL sp_get_employees();
```

Runs the procedure.

PANEL 3 — Beginner Level (Input Parameter)

Suppose HR wants employees from a specific department.

Create Procedure

```
DELIMITER $$
```

```
CREATE PROCEDURE sp_get_department_employees
```

```
(  
  IN p_department VARCHAR(50)  
)  
BEGIN  
  SELECT *  
  FROM employees  
  WHERE department = p_department;  
END $$  
  
DELIMITER ;
```

Execute

CALL sp_get_department_employees('IT');

Output

John
David

Understanding IN Parameter

IN p_department VARCHAR(50)

User passes value:

'IT'

Procedure receives it in:

p_department

PANEL 4 — Intermediate Level (Output Parameter)

Find employee count.

Create Procedure

DELIMITER \$\$

```
CREATE PROCEDURE sp_employee_count
(
    OUT total_employees INT
)
BEGIN
    SELECT COUNT(*)
    INTO total_employees
    FROM employees;
END $$

DELIMITER ;
```

Execute

```
CALL sp_employee_count(@count);
```

```
SELECT @count;
```

Output

```
4
```

Understanding OUT

Procedure sends value back.

```
OUT total_employees INT
```

Stores count into:

```
@count
```

PANEL 5 — Intermediate Level (Input + Output)

Get average salary of a department.

```
DELIMITER $$
```

```
CREATE PROCEDURE sp_avg_salary
```

```
(
  IN p_department VARCHAR(50),
  OUT avg_sal DECIMAL(10,2)
)
BEGIN
  SELECT AVG(salary)
  INTO avg_sal
  FROM employees
  WHERE department = p_department;
END $$

DELIMITER ;
```

Execute

```
CALL sp_avg_salary('IT',@avg_salary);

SELECT @avg_salary;
```

Output

60000

Because:

$(50000 + 70000)/2$

PANEL 6 — Advanced Level (Insert Employee)

Create Procedure

```
DELIMITER $$

CREATE PROCEDURE sp_add_employee
(
  IN p_name VARCHAR(100),
  IN p_department VARCHAR(50),
  IN p_salary DECIMAL(10,2)
)
BEGIN
```

```
INSERT INTO employees
(
    emp_name,
    department,
    salary
)
VALUES
(
    p_name,
    p_department,
    p_salary
);
END $$

DELIMITER ;
```

Execute

```
CALL sp_add_employee
(
    'Robert',
    'Sales',
    90000
);
```

Verify

```
SELECT *
FROM employees;
```

PANEL 7 — Advanced Level (Validation)

Allow insert only if salary > 0.

```
DELIMITER $$

CREATE PROCEDURE sp_add_employee_validation
(
    IN p_name VARCHAR(100),
    IN p_department VARCHAR(50),
    IN p_salary DECIMAL(10,2)
```

```

)
BEGIN

    IF p_salary <= 0 THEN

        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT='Salary must be greater than zero';

    ELSE

        INSERT INTO employees
        (
            emp_name,
            department,
            salary
        )
        VALUES
        (
            p_name,
            p_department,
            p_salary
        );

    END IF;

END $$

DELIMITER ;

```

Execute

```

CALL sp_add_employee_validation
(
    'Tom',
    'IT',
    -500
);

```

Error

Salary must be greater than zero

PART 2 — TRIGGERS

PANEL 8 — Beginner Level

What is a Trigger?

A trigger runs automatically when:

INSERT
UPDATE
DELETE

happens.

No CALL statement needed.

PANEL 9 — Create Audit Table

We need a table to store logs.

```
CREATE TABLE employee_audit  
(  
    audit_id INT PRIMARY KEY AUTO_INCREMENT,  
    emp_id INT,  
    action_type VARCHAR(20),  
    action_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

PANEL 10 — Beginner Trigger (AFTER INSERT)

Whenever a new employee is inserted,
automatically write a log.

Create Trigger

DELIMITER \$\$

```
CREATE TRIGGER trg_employee_insert
AFTER INSERT
ON employees
FOR EACH ROW
BEGIN
```

```
    INSERT INTO employee_audit
    (
        emp_id,
        action_type
    )
    VALUES
    (
        NEW.emp_id,
        'INSERT'
    );
```

END \$\$

DELIMITER ;

Test

```
INSERT INTO employees
(
    emp_name,
    department,
    salary
)
VALUES
(
    'Sam',
    'IT',
    65000
);
```

Check Audit

```
SELECT *
FROM employee_audit;
```

Output:

5 INSERT

What is NEW?

NEW.emp_id

means:

New row being inserted

PANEL 11 — Intermediate Trigger (AFTER UPDATE)

Track updates.

DELIMITER \$\$

```
CREATE TRIGGER trg_employee_update
AFTER UPDATE
ON employees
FOR EACH ROW
BEGIN
```

```
    INSERT INTO employee_audit
    (
        emp_id,
        action_type
    )
    VALUES
    (
        NEW.emp_id,
        'UPDATE'
    );
```

```
END $$
```

```
DELIMITER ;
```

Test

```
UPDATE employees  
SET salary=95000  
WHERE emp_id=1;
```

Audit

```
SELECT *  
FROM employee_audit;
```

Output:

```
1 UPDATE
```

PANEL 12 — Intermediate Trigger (AFTER DELETE)

DELIMITER \$\$

```
CREATE TRIGGER trg_employee_delete  
AFTER DELETE  
ON employees  
FOR EACH ROW  
BEGIN
```

```
    INSERT INTO employee_audit  
    (  
        emp_id,  
        action_type  
    )  
    VALUES  
    (  
        OLD.emp_id,  
        'DELETE'  
    );
```

```
END $$
```

```
DELIMITER ;
```

Test

```
DELETE  
FROM employees  
WHERE emp_id=2;
```

What is OLD?

Before deletion:

Mary
HR
60000

After deletion row disappears.

Trigger accesses:

OLD.emp_id

which is the row before deletion.

PANEL 13 — Advanced Trigger (Validation)

Prevent negative salary.

DELIMITER \$\$

```
CREATE TRIGGER trg_salary_validation  
BEFORE INSERT  
ON employees  
FOR EACH ROW  
BEGIN
```

```
    IF NEW.salary <= 0 THEN
```

```
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT =  
        'Salary cannot be negative';
```

```
END IF;
```

```
END $$
```

```
DELIMITER ;
```

Test

```
INSERT INTO employees
```

```
(  
  emp_name,  
  department,  
  salary
```

```
)
```

```
VALUES
```

```
(  
  'Alex',  
  'IT',  
  -100
```

```
);
```

Output:

Salary cannot be negative

PANEL 14 — Advanced Trigger (Auto Timestamp)

```
CREATE TABLE employee_history
```

```
(  
  emp_id INT,  
  old_salary DECIMAL(10,2),  
  new_salary DECIMAL(10,2),  
  modified_at DATETIME  
);
```

Trigger:

```
DELIMITER $$
```

```
CREATE TRIGGER trg_salary_change
AFTER UPDATE
ON employees
FOR EACH ROW
BEGIN
```

```
    IF OLD.salary <> NEW.salary THEN
```

```
        INSERT INTO employee_history
```

```
        (
            emp_id,
            old_salary,
            new_salary,
            modified_at
```

```
        )
```

```
VALUES
```

```
        (
            OLD.emp_id,
            OLD.salary,
            NEW.salary,
            NOW()
```

```
        );
```

```
    END IF;
```

```
END $$
```

```
DELIMITER ;
```

Test

```
UPDATE employees
SET salary = 100000
WHERE emp_id = 1;
```

Output

```
SELECT *
FROM employee_history;
1 95000 100000 2026-05-29
```

PANEL 15 — Interview Questions

Difference Between Procedure and Trigger

Procedure	Trigger
Explicitly called	Automatically executed
Can have parameters	Cannot have parameters
Returns data	Usually performs actions
Used by application	Used by database events

Types of Triggers

BEFORE INSERT
AFTER INSERT

BEFORE UPDATE
AFTER UPDATE

BEFORE DELETE
AFTER DELETE

Special Keywords

Keyword	Meaning
NEW	New row value
OLD	Existing row value
SIGNAL	Raise custom error
CALL	Execute procedure
DELIMITER	Change statement terminator

Real-World Usage

Stored Procedures

- ETL Processing
- Data Warehouse Loading
- Monthly Reports
- Payroll Processing
- Bulk Data Updates

Triggers

- Audit Logging
- Change Data Capture (CDC)
- Validation Rules
- History Tracking
- Automatic Notifications

This progression (Simple → Intermediate → Advanced) is typically enough for fresher interviews, MySQL certification preparation, and Data Engineering/Data Warehouse projects.